ANUBHAV KUMAR

Security Engineer | Backend Engineer | AI/ML Engineer
Bengaluru, Karnataka, India | +91 95230 10199 | 23btrct015@jainuniversity.ac.in
GitHub: github.com/Anubhav104401 | LinkedIn: linkedin.com/in/anubhav-kumar-b1b463298 | Watch my video intro

PROFESSIONAL SUMMARY

Computer Science undergraduate (B.Tech, 2027) who builds production backend and AI systems and then reviews them adversarially. Authored the security and governance layer of AegisProxy, an LLM gateway: constant-time credential comparison, a ten-pattern PII and PAN redactor with Luhn validation, an atomic Redis rate limiter, per-tenant budget caps and a circuit breaker — then published a findings register documenting the prompt-injection surfaces, information-disclosure paths and incomplete redaction rules still present in that same codebase. Earned a Special Mention in Caterpillar's hackathon recruitment for Cat FleetIQ, a fleet control tower with 65 REST endpoints and a rules-plus-anomaly detection engine. Seeking Security Engineer roles.

EDUCATION

Jain University, Bengaluru — B.Tech, Computer Science and Engineering 2023 – 2027

TECHNICAL SKILLS

Security Engineering: Secure Code Review, Threat Modeling, Timing-Attack Mitigation (constant-time comparison), PII and PAN Redaction (DLP) with Luhn Validation, Prompt-Injection Surface Analysis, SQL Injection Prevention, Header Injection, Insecure Deserialization, Trust Boundary Analysis, Fail-Safe Defaults, Least-Privilege Gates, Rate Limiting (Redis Lua token bucket), Circuit Breakers, Anomaly Detection (Isolation Forest), Findings Reporting with Severity Classification

Languages: Java, Python, C, JavaScript, TypeScript, SQL, Lua

AI / Machine Learning: Scikit-learn, TensorFlow, NumPy, Pandas, KMeans, PCA, Isolation Forest, Random Forest, Gradient Boosting, Support Vector Machines, TF-IDF, Natural Language Processing, SHAP and LIME (Explainable AI), Linear Programming (PuLP)

LLM Engineering: LiteLLM, Model Context Protocol (MCP), OpenAI and Google Gemini APIs, Semantic Caching, Vector Search (FAISS), Sentence Transformers, Context Compression, Prompt Engineering

Backend, Data and DevOps: FastAPI, REST API Design, async SQLAlchemy, WebSockets, Redis, SQLite, Pydantic, GraphQL, API Gateway Design, React, Next.js, Tailwind CSS, Hadoop, Hive, ETL Pipelines, Star-Schema Warehousing, Docker, Docker Compose, Git, GitHub Actions (CI/CD), pytest, Linux

Core Computer Science: Data Structures and Algorithms, Object-Oriented Programming, Operating Systems, Computer Networks, DBMS

PROJECTS

AegisProxy — LLM Gateway with Security and Governance Layer | *GitHub*

Tech: Python 3.11, FastAPI, LiteLLM, Redis, Lua, FAISS, Sentence Transformers, SQLite, Docker, pytest, GitHub Actions

- Authored the authentication layer: constant-time API key comparison via *compare_digest*, defeating byte-by-byte key recovery through a timing side channel, with separate read and write authorization gates so the observability surface — which carries prompt text — locks down automatically once an admin key signals the deployment is exposed.
- Built a ten-pattern PII and PAN redactor ordered most-specific-first so an email is consumed before bare digit patterns can fragment it, with a Luhn checksum keeping order IDs out of the card bucket, per-type hit counts surfaced to operators, and redaction enabled by default so raw prompts are never persisted without an explicit opt-out.
- Enforced abuse controls at the gateway chokepoint: an atomic Redis Lua token bucket closing the check-and-decrement race that separate GET and SET would leave open, per-tenant budget caps, a three-state circuit breaker filtering dead providers before the call, correlation IDs bounded at 128 characters against header injection, and closed-set Prometheus labels so a hostile prompt cannot explode metric cardinality.
- Published a findings register against my own codebase with exploitability, blast radius and remediation per item: verbatim provider errors that can echo prompt content, an unfenced transcript in the summariser as a live prompt-injection surface, and a PRIVATE_KEY pattern matching only the header line that leaves the key body exposed.
- Routed 100+ LLM providers behind an OpenAI-compatible API via LiteLLM, cutting repeat-query latency from a 500ms–2s upstream round trip to under 30ms with a FAISS and Redis semantic cache over all-MiniLM-L6-v2 embeddings above 0.92 cosine similarity; 389 passing tests, ruff linting and GitHub Actions CI.

Cat FleetIQ — Rental Fleet Control Tower | *Caterpillar Hackathon, Special Mention* · *GitHub*

Tech: Next.js 16, React 19, TypeScript, FastAPI, async SQLAlchemy 2.0, WebSockets, Scikit-learn, PuLP, Docker

- Delivered 65 REST endpoints across 12 routers over a 15-table async SQLAlchemy database, with Pydantic response models bounding every payload so undeclared fields cannot leak, parameterized ORM queries throughout, an explicit CORS origin allow-list, and one shared WebSocket channel pushing live updates to every panel without polling.
- Built a two-layer detection engine — an explainable rule baseline with an Isolation Forest catching what the rules miss, each alert carrying the reason it fired — alongside Holt's linear-trend forecasting with confidence bands and a PuLP solver redeploying idle equipment against forecast shortages, across nine ML and optimization modules.
- Implemented the notification service on real SMTP (smtplib with STARTTLS) with delivery logging, message preview and dry-run mode, so alerts reach live inboxes rather than a stub. Team project, placing directly after the runner-up.

TruthLens — Hybrid Fact-Verification and Fake-News Detection System | *GitHub*

Tech: Python, FastAPI, React, Vite, Scikit-learn, TF-IDF, NLTK, LIME, Google Gemini API, JWT

- Built a three-way hybrid engine fusing a Scikit-learn classifier, a rule engine scoring clickbait triggers, all-caps ratio and punctuation spikes, and a Gemini live web-verification layer over Google Search Grounding, combined by a confidence-weighted ensemble.
- Trained on 53,847 labeled articles to 90.64% accuracy over a 10-stage NLTK pipeline, adding writing-style features beyond TF-IDF — capitalization ratio, sentence length, punctuation and URL counts — to catch signals a bag-of-words model misses.
- Returned a LIME token-level explanation and cited sources with every verdict so no classification is unauditable; served over FastAPI behind JWT auth with a React dashboard.

ACHIEVEMENTS AND TECHNICAL WRITING

- Audited my own IPL match-prediction model and found target leakage — one feature mathematically identical to the label for every row — inflating held-out accuracy from a true 54.4% to 96.3%; removed it from the serving path and hard-blocked it by name against accidental reintroduction.
- Authored a severity-rated change report for AegisProxy v2.0.0 — brief, investigation, findings, remediation, verification, including the hypothesis disproved by reproduction. It surfaced a critical issue outside the brief: all five configured providers silently unreachable with no dashboard signal.
- Authored a complete technical reference for AegisProxy — request pipeline stage by stage, every module, the routing and caching algorithms, and an appendix of known defects graded by severity — plus line-by-line walkthroughs and field manuals for Cat FleetIQ and the IPL pipeline, covering the mathematics behind each module and the reasoning for every model choice over the alternatives rejected.
- Gold Badge in Java on HackerRank; 50-day LeetCode streak; optimized a Java Lomuto-partition QuickSort by removing redundant self-swaps for 8.37% faster runtime.